

Homework 7

Decision Trees & Mushroom Classification

CSCI 3202: Introduction to Artificial Intelligence

Total Points: 100 | Due: April 6 by 11:59 pm

Learning Objectives

- Fit a Decision Tree classifier using entropy
- Engineer and encode categorical features
- Handle missing data thoughtfully
- Diagnose overfitting with validation methods
- Evaluate models with precision, recall, accuracy
- Reason about real-world error consequences

Dataset

UCI Mushroom Dataset

8,124 mushroom specimens
22 categorical features
Binary target: edible (e) or poisonous (p)
Source: UCI ML Repository
URL: archive.ics.uci.edu/ml/datasets/mushroom

Overview

In this assignment, you will build a Decision Tree classifier to predict whether a mushroom is edible or poisonous using the classic UCI Mushroom Dataset. This is a high-stakes classification problem: a false negative (predicting edible when the mushroom is actually poisonous) could have fatal consequences. You will practice the full machine learning workflow — from data loading and feature engineering, through model training, to evaluation and prediction.

You are provided with a starter Jupyter Notebook (hw7_starter.ipynb) that contains the skeleton code. You must complete all sections marked with # YOUR CODE HERE and write thorough answers to all written questions.

Part 1: Data Loading and Exploration (15 points)

1.1 Load the Dataset (5 points)

Load the UCI Mushroom dataset using the pandas library. The dataset can be fetched directly using the ucimlrepo Python package (see starter notebook) or downloaded as agaricus-lepiota.data from the UCI repository.

- Load the data into a pandas DataFrame and display the first 10 rows.
- Print the shape of the DataFrame (rows and columns).
- Display a summary of all columns, including their data types.
- Report the class distribution: how many edible vs. poisonous mushrooms are in the dataset? Is the dataset balanced or imbalanced?

1.2 Missing Data Investigation (10 points)

The UCI Mushroom dataset encodes missing values as the character '?' in one of the feature columns.

- Identify which column(s) contain missing values represented as '?'. Report the count of missing values per column.
- Describe two different strategies you could use to handle these missing values (e.g., imputation, dropping rows/columns, treating '?' as its own category).
- Justify which strategy you will use and implement it. Explain any trade-offs of your chosen approach.

Written Question 1.2 (5 pts): Suppose the missing values are not random — for example, suppose field collectors systematically skip recording a measurement when a mushroom has a certain characteristic. How might this introduce bias into your model? What could go wrong?

Part 2: Feature Engineering (20 points)

2.1 Encoding Categorical Features (10 points)

All 22 features in this dataset are categorical (represented as single letters). Decision trees in scikit-learn require numerical input.

- Apply Label Encoding or One-Hot Encoding to convert all categorical features into numerical form.
- Display the transformed DataFrame to confirm the encoding was applied correctly.
- Report the final shape of your feature matrix X and target vector y after encoding.

2.2 Train/Test Split (10 points)

- Split your data into training (80%) and test (20%) sets using `train_test_split` with a fixed `random_state` for reproducibility.
- Verify the split by printing the shapes of `X_train`, `X_test`, `y_train`, and `y_test`.
- Check that the class distribution (edible vs. poisonous) is similar in both the training and test sets. Use the `stratify` parameter if needed.

Written Question 2.2 (5 pts): Why is it important to split data into training and test sets before model evaluation? What could go wrong if you evaluated your model on the same data you trained it on?

Part 3: Training a Decision Tree with Entropy (20 points)

3.1 Fit the Model (10 points)

Using scikit-learn's DecisionTreeClassifier, train a decision tree on the training data.

- Set criterion='entropy' to use information gain for splitting.
- Start without constraining the tree depth (max_depth=None).
- Print the depth and number of leaves of the trained tree.

Written Question 3.1 (5 pts): In your own words, explain what entropy means in the context of a decision tree. What does it mean for a node to have entropy = 0? What does entropy = 1 mean (for a binary classification problem)? Why does minimizing entropy at each split lead to better classification?

3.2 Visualizing the Tree (10 points)

- Use sklearn.tree.plot_tree or export_text to visualize the first 3 levels of your trained tree.
- Identify and label the root node. What feature does the model split on first? Why do you think this feature was chosen?
- For at least one internal node in your visualization, compute the entropy of the samples at that node by hand (showing your work). Verify your result matches what the tree shows.

Part 4: Overfitting and Model Selection (20 points)

4.1 Detecting Overfitting (10 points)

An unconstrained decision tree often overfits the training data — memorizing it rather than learning general patterns.

- Compute and report training accuracy and test accuracy for your unconstrained tree.
- Train a series of trees with max_depth ranging from 1 to 20. For each depth, record the training accuracy and test accuracy.
- Plot training accuracy and test accuracy vs. max_depth on the same graph. Label axes and include a legend.

Written Question 4.1 (5 pts): Looking at your plot, at what tree depth does overfitting appear to begin? How can you tell from the graph? Describe the general shapes of the training and test accuracy curves and explain what each pattern indicates about model complexity.

4.2 Cross-Validation (10 points)

- Use 5-fold cross-validation (cross_val_score) on the training set to evaluate your model at max_depth=5 and max_depth=None.
- Report the mean and standard deviation of cross-validation accuracy for each depth setting.

- Based on your cross-validation results, select the best `max_depth` for your final model and justify your choice.

Part 5: Model Evaluation (15 points)

5.1 Goodness-of-Fit Metrics (10 points)

Using your best model (chosen in Part 4), evaluate performance on the held-out test set.

- Compute and report: Accuracy, Precision, Recall, F1 Score.
- Generate and display the confusion matrix. Label all four quadrants: True Positive (TP), False Positive (FP), True Negative (TN), False Negative (FN).
- Plot the confusion matrix as a heatmap using `seaborn` or `matplotlib`.

5.2 Interpreting Errors (5 points)

For this mushroom classification task, define: Positive class = Poisonous, Negative class = Edible.

Written Question 5.2a (3 pts): A False Positive means your model predicted a mushroom is poisonous when it is actually edible. A False Negative means your model predicted a mushroom is edible when it is actually poisonous. Describe the real-world consequences of each type of error. Which error is more dangerous in this domain? Which metric — precision or recall — should be prioritized, and why?

Written Question 5.2b (2 pts): Imagine you are deploying this model to a foraging app used by hikers. A friend who disagrees with you argues that precision is the most important metric. Construct a counterargument explaining why recall should be prioritized in this life-safety context. Use specific numbers from your confusion matrix to support your argument.

Part 6: Prediction Function (10 points)

6.1 Build a Prediction Function (10 points)

Write a Python function called `predict_mushroom(features)` that takes a dictionary of raw mushroom features as input and returns a human-readable prediction: 'Edible' or 'Poisonous'.

Your function must: (1) accept a dictionary where keys are feature names and values are the raw categorical codes (e.g., `{'cap-shape': 'x', 'cap-color': 'n', ...}`), (2) apply the same encoding used during training, (3) use your trained model to generate a prediction, and (4) return the string 'Edible' or 'Poisonous'.

Apply your function to the following three mushroom samples and report the prediction for each:

Sample	cap-shape	cap-color	odor	gill-color	spore-print-color
Mushroom A	x (convex)	n (brown)	a (almond)	k (black)	n (brown)
Mushroom B	f (flat)	y (yellow)	f (foul)	b (buff)	w (white)
Mushroom C	b (bell)	w (white)	n (none)	w (white)	w (white)

Note: For features not listed in the table above, use the most common value (mode) from the training set. Document your approach in your notebook.

Submission Instructions

- Submit one Jupyter Notebook (.ipynb) with all cells executed and outputs visible.
- All written question responses should be in Markdown cells immediately following the relevant code cells.
- Use clear section headers matching the parts above.
- Your code must run top-to-bottom without errors when 'Restart and Run All' is executed.

Grading Rubric

Section	Points	Key Criteria
Part 1: Data Loading & Exploration	15	Correct load, shape/type report, class balance, missing data identified and handled with justification
Part 2: Feature Engineering	20	Correct encoding, final shapes reported, train/test split with stratification, written questions answered
Part 3: Decision Tree Training	20	Entropy criterion used, tree depth/leaves reported, tree visualized, entropy computed by hand for one node
Part 4: Overfitting & Model Selection	20	Train vs. test accuracy plotted, overfitting identified, CV performed, best depth justified
Part 5: Model Evaluation	15	All 4 metrics computed, confusion matrix labeled correctly, FP/FN consequences clearly reasoned

Part 6: Prediction Function	10	Function handles encoding, uses trained model, returns correct string, all 3 samples predicted
Total	100	Code runs cleanly; all questions answered in Markdown

Academic Integrity: All code and written responses must be your own. You may use course notes, the scikit-learn documentation, and pandas documentation. Do not share code with classmates or use AI code-generation tools. Suspected violations will be reported to the academic integrity office.